

☺ SSD 自动化测试脚本说明文档

1. 一、项目概述

本脚本面向 Linux (Ubuntu) 平台下的 SSD 测试工程师，覆盖以下八大测试项目：

序号	测试项目	英文名称	核心目标
1	现场固件升级/降级	Field Firmware Upgrade/Downgrade	验证固件正常烧写、升降级兼容性、烧写后无掉盘/崩溃/功能异常，版本变更后功能正常
2	设备智能健康信息	Device SMART Health Information	确认 OS 枚举 SSD，获取完整 SMART，检查电源循环/通电时间/温度/可用备件/介质错误，R/W 后复检 SMART
3	设备容量	Devices capacity	通过 nvme-cli / 磁盘工具读取并校验容量
4	完整性能特征	Full Performance Characterization	FOB（出厂空白）与稳态两种状态下测量带宽、IOPS、延迟、QoS
5	读/写测试	Read/Write Test	全磁盘写入+读取验证、多文件大小（256MB/1GB/4GB/16GB/32GB）重复读写周期、24小时长期读写验证，数据完整性校验无丢失/损坏/比特错误
6	正常电源循环测试	Normal Power Cycle Test	持续混合读写下正常关机→断电→开机，循环检查磁盘/分区/文件系统/数据完整性，最终完整功能测试

Table 1

序号	测试项目	英文名称	核心目标
7	意外电源循环测试	Surprise Power Cycle Test (SPOR)	SSD读写过程中直接硬件断电（不待机、不sync），验证PLP掉电保护对FTL元数据的备份能力，上电后数据完整性校验
8	操作系统中断测试	OS Interruption Test	验证SSD在持续I/O期间S3/S4休眠/唤醒的稳定性与数据完整性，循环检查磁盘标识/分区/数据，长期循环验证

脚本语言：Python 3，依赖标准 Linux 工具链（nvme-cli、smartmontools、fio、ipmitool、pyserial、rtcwake）。

双模式运行：

- 命令行模式： `sudo python3 ssd_test.py -d /dev/nvme0n1 -t all -y`，适合自动化/CI 集成
- 可视化上位机模式（GUI）： `sudo python3 ssd_test.py --gui`，基于 tkinter 标准库，图形化选择设备/测试项/参数，实时日志和进度显示，支持配置保存/加载

二、运行环境准备

2.1 系统要求

- 操作系统：Ubuntu 20.04 / 22.04 / 24.04 LTS（x86_64 或 arm64）
- 内核版本：≥ 5.4（NVMe 驱动稳定）
- Python：≥ 3.8
- 权限：需要 root 或 sudo 权限（操作块设备、安装固件、fio 直写）
- 待测 SSD：NVMe SSD（/dev/nvmeXnY）或 SATA SSD（/dev/sdX）

2.2 系统工具安装

```
1 # 更新软件源
2 sudo apt update
3
4 # 安装 NVMe 命令行工具（固件升级、SMART、容量读取核心依赖）
5 sudo apt install -y nvme-cli
6
7 # 安装 SMART 监控工具（SATA/NVMe 通用 SMART 读取）
8 sudo apt install -y smartmontools
9
10 # 安装性能测试工具（FIO/稳态性能、带宽、IOPS、延迟、QoS）
11 sudo apt install -y fio
12
```

```
13 # 安装块设备信息工具 (lsblk、blkdiscard 等, 通常系统自带)
14 sudo apt install -y util-linux
15
16 # 安装 IPMI 工具 (电源循环测试的远程电源控制依赖, 服务器环境)
17 sudo apt install -y ipmitool
18
19 # 安装文件系统校验工具 (电源循环测试的数据完整性校验依赖)
20 sudo apt install -y e2fsprogs parted
21
22 # 安装 Python 串口库 (SPOR 测试 Timeboard 硬件断电控制依赖)
23 pip3 install pyserial
24
25 # 配置串口权限 (Timeboard 通常使用 /dev/ttyUSB0)
26 sudo usermod -aG dialout $USER
27 sudo chmod 666 /dev/ttyUSB0 2>/dev/null || true
28
29 # 安装 JSON 处理工具 (可选, 用于手动校验输出)
30 sudo apt install -y jq
```

Fence 1

验证安装:

```
1 nvme --version      # 应输出 nvme-cli 版本
2 smartctl --version  # 应输出 smartmontools 版本
3 fio --version       # 应输出 fio 版本
4 ipmitool -V         # 应输出 ipmitool 版本 (电源循环测试需要)
5 python3 --version   # 应输出 Python 3.8+
```

Fence 2

2.3 Python 依赖

脚本仅使用 Python 标准库 (subprocess、json、argparse、logging、datetime、os、sys、re、time、statistics), 无需额外 pip 安装。

如需生成 HTML 报告 (可选扩展), 可安装:

```
1 | pip3 install jinja2
```

Fence 3

2.4 待测设备确认

```
1 # 列出所有 NVMe 设备
2 sudo nvme list
3
4 # 列出所有块设备 (含 SATA)
5 lsblk -d -o NAME,SIZE,MODEL,TRAN
6
7 # 确认设备未挂载 (测试前必须卸载所有分区)
8 sudo umount /dev/nvme0n1p* 2>/dev/null || true
```

Fence 4

警告: 性能测试和固件升级会破坏待测 SSD 上的所有数据, 请确保设备上无重要数据, 或已完成备份。

3. 三、脚本制作过程

3.1 3.1 整体架构

脚本采用模块化设计，单一入口 `ssd_test.py`，内部按测试项目拆分为独立类/函数：

```
1  ssd_test.py
2  |— 配置与日志模块
3  |   |— setup_logging()      # 日志初始化（控制台 + 文件）
4  |   |— TestConfig          # 测试参数配置（运行时长、队列深度等）
5  |
6  |— 工具函数模块
7  |   |— run_cmd()            # 统一命令执行（超时、重试、错误捕获）
8  |   |— check_root()        # root 权限校验
9  |   |— check_dependencies() # 依赖工具存在性检查
10 |   |— get_device_type()    # 识别 NVMe / SATA 设备类型
11 |   |— bytes_to_human()     # 字节数人性化转换
12 |
13 |— 测试项 1: 固件升级/降级
14 |   |— FirmwareTester
15 |       |— get_current_fw_version() # 读取当前固件版本
16 |       |— fw_download()            # 下载固件到设备
17 |       |— fw_commit()              # 提交/激活固件（含复位）
18 |       |— verify_device_after_flash() # 烧写后设备枚举与功能校验
19 |       |— run_upgrade()            # 执行升级流程
20 |       |— run_downgrade()          # 执行降级流程
21 |
22 |— 测试项 2: SMART 健康信息
23 |   |— SmartTester
24 |       |— get_smart_nvme()         # nvme smart-log 获取 SMART
25 |       |— get_smart_smartctl()     # smartctl 获取 SMART
26 |       |— check_smart_core_items() # 核心项检查（电源循环/通电时间/温度/可用备件/介质错误）
27 |       |— run_basic_rw()           # 基本读写操作
28 |       |— run_test()               # SMART 测试主流程（含 R/W 后复检）
29 |
30 |— 测试项 3: 设备容量
31 |   |— CapacityTester
32 |       |— get_capacity_nvme()       # nvme id-ctrl 读取容量
33 |       |— get_capacity_lsblk()      # lsblk 读取容量
34 |       |— verify_capacity()         # 多源容量交叉校验
35 |       |— run_test()               # 容量测试主流程
36 |
37 |— 测试项 4: 完整性能特征
38 |   |— PerformanceTester
39 |       |— precondition_steady_state() # 稳态预处理（顺序写满 + 随机写）
40 |       |— run_fio()                  # 执行单次 fio 测试
41 |       |— test_sequential_read_write() # 顺序读写带宽测试
42 |       |— test_random_read_write()    # 随机读写 IOPS/延迟测试
43 |       |— test_latency_qos()          # 延迟与 QoS 百分位测试
44 |       |— run_fob_test()              # FOB 状态性能测试
45 |       |— run_steady_state_test()     # 稳态性能测试
46 |
47 |— 测试项 5: 读/写测试
48 |   |— ReadWriteTester
49 |       |— _check_device_online()     # 设备在线/掉盘检测
50 |       |— _get_device_size_gb()      # 获取设备容量
51 |       |— _check_smart()             # SMART 健康检查
52 |       |— _get_pattern_arg()         # fio 数据 pattern 参数
53 |       |— run_full_disk_test()       # 全磁盘写入+读取验证（fio verify）
54 |       |— run_file_cycle_test()      # 多文件大小重复读写周期
55 |       |— run_long_run_test()        # 24小时长期读写验证
56 |       |— run()                     # 读/写测试主入口
57 |
58 |— 测试项 6: 正常电源循环测试
59 |   |— PowerCycleTester
60 |       |— setup_test_partition()     # 创建测试分区与文件系统
61 |       |— write_integrity_data()     # 写入校验数据（带校验和）
```

```

62 |         | verify_integrity_data()          # 校验数据完整性
63 |         | start_mixed_rw_load()          # 启动持续混合读写负载 (fio 后台)
64 |         | stop_mixed_rw_load()          # 停止混合读写负载
65 |         | ipmi_power_off()              # IPMI 远程断电
66 |         | ipmi_power_on()              # IPMI 远程上电
67 |         | ipmi_power_status()           # IPMI 电源状态查询
68 |         | graceful_shutdown()           # 正常关机
69 |         | check_disk_after_boot()       # 开机后磁盘/分区/文件系统/数据检查
70 |         | load_state() / save_state()   # 状态文件持久化 (重启后恢复)
71 |         | run_final_functional_test()   # 最终完整功能测试
72 |         | run()                         # 电源循环测试主流程
73 |
74 | └─ 测试项 7: 意外电源循环测试 (SPOR)
75 |     | └─ TimeboardController            # Timeboard 硬件断电控制器 (Modbus RTU)
76 |     |     | └─ modbus_crc16()           # Modbus CRC16 校验
77 |     |     | └─ build_modbus_rtu_packet() # 组装 0x10 写寄存器报文
78 |     |     | └─ connect() / disconnect() # 串口连接/断开 (自动探测 ttyUSB/ACM)
79 |     |     | └─ trigger_poweroff()       # 触发延时硬件断电
80 |     | └─ SPORTester
81 |     |     | └─ load_state() / save_state() # 原子状态文件 (临时文件+fsync+rename)
82 |     |     | └─ check_device_present()    # 掉盘检测 (上电后设备是否枚举)
83 |     |     | └─ check_smart()             # SMART 健康检查 (介质错误/可用备件/温度)
84 |     |     | └─ check_pcie_link()         # PCIe 链路状态检查 (按 0108 类代码)
85 |     |     | └─ write_pattern()           # 写入 pattern 打底 (fio do_verify+crc32c)
86 |     |     | └─ verify_pattern()          # 验证 pattern (fio buffer_pattern 比对)
87 |     |     | └─ start_spor_write()        # 启动 SPOR 写入 (高 iodepth, 支持混合读写)
88 |     |     | └─ get_write_position()      # 从 fio iolog 解析掉电时最后写入 LBA
89 |     |     | └─ phase1_poweroff()         # 阶段1: 掉电前 (不 sync, 直接硬件断电)
90 |     |     | └─ phase2_poweron()          # 阶段2: 上电后 (掉盘+SMART+前段/后段验证)
91 |     |     | └─ run_final_functional_test() # 最终完整功能测试 (容量+SMART+顺序读写性能)
92 |     |     | └─ print_final_results()     # 打印最终结果汇总
93 |     |     | └─ run()                     # SPOR 测试主入口 (两阶段状态机)
94 |
95 | └─ 测试项 8: 操作系统中断测试 (OSINT)
96 |     | └─ OSInterruptTester
97 |     |     | └─ load_state() / save_state() # 原子状态文件持久化
98 |     |     | └─ setup_test_partition()     # 创建分区+ext4文件系统
99 |     |     | └─ write_integrity_data()     # 写入带SHA-256校验和的测试文件
100 |     |     | └─ verify_integrity_data()    # 校验数据完整性 (SHA-256比对)
101 |     |     | └─ start_io_load() / stop_io_load() # fio混合读写持续负载 (后台)
102 |     |     | └─ trigger_sleep()            # 触发S3/S4休眠 (rtctime定时唤醒)
103 |     |     | └─ check_disk_identity()      # 磁盘标识检查 (/dev/disk/by-id/)
104 |     |     | └─ check_partition_and_fs()   # 分区表+文件系统+fsck检查
105 |     |     | └─ check_smart()              # SMART健康检查
106 |     |     | └─ run_cycle()                # 单轮循环 (IO+休眠+唤醒检查)
107 |     |     | └─ run_final_functional_test() # 最终完整功能测试
108 |     |     | └─ print_final_results()      # 打印最终结果汇总
109 |     |     | └─ run()                     # OSINT测试主入口
110 |
111 | └─ 报告模块
112 |     | └─ TestResult                      # 单条测试结果数据结构
113 |     | └─ TestReport                      # 报告汇总 (PASS/FAIL 统计)
114 |     | └─ generate_json_report()          # 生成 JSON 报告
115 |     | └─ generate_text_summary()         # 生成文本摘要
116 |
117 | └─ 可视化上位机 (GUI)
118 |     | └─ SSDTestGUI
119 |     |     | └─ _build_menu()             # 菜单栏 (文件/工具/帮助)
120 |     |     | └─ _build_device_panel()     # 设备选择面板 (扫描+下拉+设备信息)
121 |     |     | └─ _build_test_items_panel() # 测试项选择面板 (7项复选框+全选/清空)
122 |     |     | └─ _build_params_panel()     # 参数配置面板 (Notebook 6个标签页)
123 |     |     | └─ _build_log_panel()        # 日志输出面板 (彩色分级+自动滚动)
124 |     |     | └─ scan_devices()            # 扫描系统存储设备 (lsblk+nvm list)
125 |     |     | └─ build_command()           # 根据GUI配置构建命令行参数
126 |     |     | └─ start_test()              # 开始测试 (subprocess.Popen)
127 |     |     | └─ stop_test()               # 停止测试 (terminate/kill)
128 |     |     | └─ _read_output()            # 后台线程读取子进程输出
129 |     |     | └─ _append_log()             # 追加彩色日志
130 |     |     | └─ save_config()/load_config() # 配置保存/加载 (JSON)
131 |     |     | └─ export_log()              # 导出日志到文件
132 |     |     | └─ run_gui()                 # GUI 启动入口
133 |
134 | └─ 主入口
135 |     | └─ parse_args()                    # 命令行参数解析

```

3.2 3.2 关键实现思路

3.2.1 3.2.1 命令执行封装

所有外部命令通过 `run_cmd()` 统一执行，具备：

- 超时控制（默认 300s，fio 测试可配置）
- 实时输出捕获（stdout/stderr 分离）
- 重试机制（瞬时错误最多重试 2 次）
- 错误码检查与异常抛出
- 命令日志记录（便于问题追溯）

3.2.2 3.2.2 固件升级/降级

NVMe 固件升级标准流程（基于 `nvme-cli`）：

1. `nvme fw-download` 将固件镜像下载到设备暂存区
2. `nvme fw-commit` 提交并激活固件（`action=2` 为立即复位激活，`action=1` 为下次复位激活）
3. 复位后等待设备重新枚举（轮询 `nvme list`，最长 120s）
4. 读取新固件版本并与目标版本比对
5. 执行基本功能校验（SMART 可读、容量正常、基本 R/W 正常）

降级流程与升级一致，仅镜像文件为旧版本。

注意：部分厂商 SSD 需要专用工具（如三星 `magician`、西数 `nvmecli`），脚本预留 `vendor_tool` 参数接口。

3.2.3 3.2.3 SMART 健康信息

- NVMe 设备优先使用 `nvme smart-log /dev/nvmeX`（输出 JSON 格式，字段标准化）
- SATA 设备或需要更多字段时使用 `smartctl -a /dev/sdX`（解析文本输出）
- 核心检查项：
 - 电源循环数（Power Cycles）：R/W 前后应增加合理值
 - 通电时间（Power On Hours）：应随测试时间增加
 - 温度（Temperature）：应在正常工作范围（0~70°C）
 - 可用备件（Available Spare / Percentage Used）：不应低于阈值
 - 介质错误（Media and Data Integrity Errors）：R/W 后不应新增
- 基本 R/W：使用 `dd` 或 `fio` 对设备前 1GB 进行读写，然后重新读取 SMART 对比变化

3.2.4 3.2.4 设备容量

- NVMe： `nvme id-ctrl /dev/nvmeX` 读取 `tnvmcap`（总 NVM 容量）和 `unvmcap`（未分配 NVM 容量）
- 通用： `lsblk -b -d -n -o SIZE /dev/nvmeXnY` 读取块设备字节数
- 交叉校验：两种方式读取的容量差值应在合理范围（< 1%，因保留空间/计算方式差异）

- 容量换算：1GB = 10^9 字节（厂商十进制），1GiB = 2^30 字节（操作系统二进制），报告中同时标注

3.2.5 3.2.5 完整性能特征

FOB 状态（Fresh Out of Box）：

- 前置条件：执行 `blkdiscard` 或 `nvme format` 清空全盘，恢复出厂空白状态
- 测试项：
 - 顺序读/写 128K QD32：测量带宽（MB/s）
 - 随机读/写 4K QD32：测量 IOPS
 - 随机读 4K QD1：测量平均延迟（μs）
 - 每项运行时长 ≥ 60s（稳态测量建议 ≥ 300s）

稳态（Steady State）：

- 预处理流程（SNIA 标准近似）：
 1. 顺序写满全盘 2 次（128K QD32）
 2. 随机写 4K QD32 持续至写入量达到 2 倍全盘容量
 3. 等待 5 分钟让后台 GC/WL 完成
- 稳态测试项与 FOB 相同，用于对比性能衰减

QoS 测量：

- fio 输出中读取延迟百分位：clat percentiles (p50/p90/p99/p99.9/p99.99)
- 报告 p99 延迟和最大延迟，评估 QoS 表现

fio 关键参数：

- `--direct=1`：绕过页缓存，直写设备
- `--ioengine=libaio`：Linux 异步 IO
- `--runtime=60`：运行时长
- `--time_based`：按时间运行（即使写完也循环）
- `--group_reporting`：汇总多 job 结果
- `--output-format=json`：JSON 输出便于解析

3.2.6 3.2.6 正常电源循环测试

测试流程（每个循环）：

1. 在待测 SSD 上创建测试分区与 ext4 文件系统
2. 写入带校验和的测试数据（用于重启后验证数据完整性）
3. 启动持续混合读写负载（fio `randrw`，读占比 70%，后台运行）
4. 正常关机（`shutdown -h now` 或 `systemctl poweroff`）
5. 断电（IPMI `chassis power off`，或手动断电）
6. 等待 30 秒（模拟真实断电间隔）
7. 上电（IPMI `chassis power on`，或手动上电）
8. 等待系统启动并 SSH/本地可登录（轮询，最长 300s）

9. 开机后检查：

- 磁盘枚举（`lsblk` / `nvme list`）
- 分区表完整性（`parted` / `fdisk`）
- 文件系统检查（`fsck`，只读模式）
- 数据完整性校验（比对校验和）
- SMART 检查（无新增介质错误、温度正常）

10. 记录本循环结果，进入下一循环

11. 达到设定循环次数后，执行最终完整功能测试（容量 + SMART + 基本性能）

状态持久化与重启恢复：

- 使用 JSON 状态文件（默认 `/var/lib/ssd_power_cycle_state.json`）记录：
 - 当前循环次数、目标循环次数
 - 每循环的检查结果
 - 测试数据校验和
 - 测试阶段标识（`setup` / `rw_load` / `shutdown` / `boot_check` / `final_test` / `done`）
- 系统重启后，脚本自动检测状态文件，从断点继续执行
- 支持通过 `systemd service` 实现开机自启，完成全自动循环

IPMI 远程电源控制（服务器环境推荐）：

- 依赖 `ipmitool`，通过 BMC 远程控制服务器电源
- 关键命令：
 - `ipmitool -H <BMC_IP> -U <user> -P <pass> chassis power status`：查询电源状态
 - `ipmitool ... chassis power off`：软关机后断电
 - `ipmitool ... chassis power on`：上电开机
 - `ipmitool ... chassis power cycle`：电源循环（off→on）
- IPMI 参数通过 `--ipmi-host`、`--ipmi-user`、`--ipmi-pass` 传入
- 无 IPMI 环境时使用 `--power-mode manual`，脚本在关机前提示用户手动断电上电

混合读写负载：

- fio 参数：`--rw=randrw --rwmixread=70 --bs=4k --iodepth=32 --numjobs=4`
- 后台运行，日志写入文件，关机前停止并统计读写量
- 模拟真实业务负载下的电源循环场景

数据完整性校验：

- 写入时生成多个测试文件（每个 100MB），每个文件附带 SHA-256 校验和
- 校验和记录在状态文件中（不存储在待测盘上，避免待测盘故障导致校验和丢失）
- 开机后重新计算校验和并比对，任何不一致即判定本循环 FAIL

3.2.7 意外电源循环测试 (SPOR)

与正常电源循环的核心区别：

- 正常电源循环：先正常关机（`shutdown`），再断电——SSD 有机会完成 FTL 刷新
- SPOR 意外断电：在 SSD **活跃读写过程中直接硬件断电**，不执行 `sync`、不发送待机命令——SSD 完全依赖内置 PLP（Power Loss Protection，掉电保护）电容供电，在毫秒级时间内完成 FTL 映射表等关键元数据的备份

测试流程（每轮两阶段）：

阶段1（掉电前）：

- 写入 pattern11（0x11）打底 20GB，使用 `fio do_verify=1 --verify=crc32c` 确保初始数据正确
- 验证 pattern11 完整性
- 启动 SPOR 写入（pattern22，0x22），使用高队列深度 `--iodepth=32 --numjobs=4` 保证写入压力；支持 `--spor-mixed-rw` 切换为混合读写（`randrw`，70%读）
- 等待指定秒数（`--spor-delay`，默认 5s），此时 `fio` 仍在活跃写入
- 不执行** `os.sync()`，直接保存状态文件，然后通过 Timeboard 触发硬件断电（`--spor-poweroff-delay-ms`，默认 500ms，越短越"意外"）

阶段2（上电后）：

- 等待 15s SSD 初始化，**掉盘检测**（设备节点是否存在）
- PCIe 链路状态检查（按 NVMe 类代码 0108 筛选）
- SMART 健康检查**（介质错误数、可用备件、温度），记录每轮 SMART 历史
- 从 `fio write_iolog` 解析掉电时最后写入 LBA 位置（读取最后 20 行，LBA 对齐校验，跳过截断条目）
- 前段验证**：0 ~ 掉电位置（跳过最后 8 个 LBA 容错，因为掉电瞬间可能未完全写入），验证 pattern22
- 后段验证**：掉电位置 ~ 20GB 末尾，验证未被覆盖的 pattern11
- 记录本轮结果（pattern22 + pattern11 + SMART 全部通过才 PASS）
- 全部循环完成后执行**最终完整功能测试**（容量 + SMART + 顺序读/写性能 30s）

PLP 有效性验证原理：

- 如果 SSD 的 PLP 正常工作，掉电瞬间电容供电会将 FTL 映射表、缓存中的脏数据写入 NAND，上电后已写入区域的数据应完整可读（pattern22 验证通过），未写入区域应保持原始 pattern11
- 如果 PLP 失效，FTL 映射表可能损坏，导致已写入 LBA 映射错乱、数据读回错误（pattern22 验证失败），甚至 SSD 掉盘（设备不枚举）
- 跳过掉电边界最后 N 个 LBA 是因为这几个 LBA 可能恰好在掉电瞬间处于"已发送写命令但数据未完全落盘"的中间状态，属于正常现象，不代表 PLP 失效

Timeboard 硬件控制：

- 通过 USB 转串口（`/dev/ttyUSB0`，自动探测 ttyUSB0-2/ttyACM0-1）连接 Timeboard 继电器电源板
- 使用 **Modbus RTU 协议**（功能码 0x10 写多个寄存器），从站地址 0x01，起始地址 0x0020，写入 3 个寄存器（6 字节数据：CMD_TYPE + CMD_LENGTH + 24bit 延时 + 补零）
- 延时单位为 100ms，硬件收到命令后延时指定时间切断 SSD 供电
- 自带 Modbus CRC16 校验（多项式 0xA001），确保命令可靠传输

状态文件原子写入：

- 状态文件路径 `/var/lib/ssd_spor_state.json`，记录当前轮数、阶段、每轮结果、SMART 历史

- 采用**原子写入**：写入临时文件 → `fsync` 文件内容 → `os.replace` 原子重命名 → `fsync` 目录项

- 确保状态文件本身不会因意外断电而损坏（半写状态），上电后可可靠恢复

关键设计决策（相比原始代码的改进）：

1. **删除断电前** `os.sync()`：原始代码在断电前执行 `sync`，会促使 SSD 刷新缓存，削弱"意外"性；移植版完全不 `sync`，在 `fio` 活跃写入时直接断电

2. **提高写入压力**：原始代码 SPOR 写入 `iodepth` 默认为 1（未设置），5 秒仅写 1.5GB；移植版设置 `--iodepth=32 --numjobs=4`，写入压力大幅提升，更能暴露 PLP 在高负载下的缺陷

3. **增加掉盘检测**：上电后先检查设备节点是否存在，明确区分"SSD 掉盘"和"数据错误"

4. **增加 SMART 检查**：每轮上电后检查介质错误数、可用备件、温度，记录历史变化，PLP 失效可能导致 NAND 写入损坏（介质错误增加）

5. **增加最终完整功能测试**：所有循环完成后执行容量 + SMART + 顺序读写性能测试，确认多轮意外断电后 SSD 功能正常、性能无衰减

6. **支持混合读写**：新增 `--spor-mixed-rw` 选项，可切换为 `randrw` 混合负载，更接近真实业务场景

7. **修复 PCIe 链路检查**：原始代码 `grep -i nvme` 可能匹配不到（`lspci` 输出为"Non-Volatile memory controller"）；移植版按 PCIe 类代码 `0108` 筛选

8. **硬件断电延时可配置**：`--spor-poweroff-delay-ms` 默认 500ms（原始代码固定 2000ms），延时越短越接近"真正意外"

3.2.8 3.2.8 操作系统中断测试 (OSINT)

与电源循环测试的核心区别：

- 正常/意外电源循环：直接切断 SSD 供电，测试 SSD 在断电场景下的稳健性

- OSINT 操作系统中断：操作系统进入 S3（挂起到内存）/ S4（挂起到磁盘/休眠）状态，SSD 收到 NVMe 待机电命令后进入低功耗状态，然后系统唤醒，SSD 恢复正常工作。测试 SSD 在操作系统休眠/唤醒切换时的稳定性和数据完整性

测试流程（每轮）：

1. 在待测 SSD 上创建 GPT 分区 + ext4 文件系统，写入 512MB 随机测试文件并计算 SHA-256 校验和（校验和存入状态文件，不存待测盘）

2. 启动 `fio` 混合读写持续负载（`randrw`，70%读，4K，`iodepth=32`，`numjobs=4`），运行指定秒数（`--osint-io-duration`，默认 60s）

3. **活跃 IO 模式**：在 `fio` 仍在读写时触发休眠；**空闲模式**（`--osint-io-idle`）：停止 IO 后空闲状态触发休眠

4. 使用 `rtcwake -m mem -s <seconds>`（S3）或 `rtcwake -m disk -s <seconds>`（S4）触发休眠并设置 RTC 闹钟定时唤醒，无需人工干预

5. 系统唤醒后等待 10s 稳定，停止 IO 负载

6. 唤醒后四项检查：

- **磁盘标识**：检查 `/dev/disk/by-id/` 符号链接是否稳定，设备节点是否存在

- **分区与文件系统**：检查 GPT 分区表、重新挂载、`fsck -n` 只读检查文件系统一致性

- **数据完整性**：重新计算测试文件 SHA-256 校验和并与原始值比对

- **SMART 健康**：检查介质错误数、可用备件、温度

7. 记录本轮结果，进入下一轮

8. 全部循环完成后执行最终完整功能测试（容量 + SMART + 顺序读/写性能 30s）

S3 vs S4 休眠：

- **S3（Suspend to RAM，挂起到内存）**：系统状态保存在内存中，CPU 停止，内存自刷新，SSD 收到 NVMe 待机命令进入低功耗。唤醒速度快（秒级），但需要持续供电保持内存数据
- **S4（Hibernate，挂起到磁盘）**：系统状态写入 swap 分区，完全断电。唤醒时从 swap 恢复，速度慢（数十秒），但完全不需要供电
- 脚本支持 `--osint-sleep-type s3/s4/both`，`both` 模式下每轮交替执行 S3 和 S4

rtcwake 定时唤醒机制：

- `rtcwake -m <mode> -s <seconds>`：设置 RTC（实时时钟）闹钟，在指定秒数后唤醒系统，然后进入指定休眠状态
- `rtcwake` 命令会阻塞直到系统唤醒，脚本流程线性执行，无需像 SPOR 那样的两阶段状态机
- 需要 root 权限，且系统 BIOS/UEFI 需支持 RTC 唤醒
- S4 休眠需要配置足够大的 swap 分区（至少大于内存大小）

数据完整性验证原理：

- 休眠前写入随机数据文件并记录 SHA-256 校验和
- 休眠/唤醒过程中，如果 SSD 固件在待机/恢复时出现异常，可能导致：
 - 磁盘标识变化（`/dev/disk/by-id/` 链接丢失）
 - 分区表损坏（GPT 头/备份头不一致）
 - 文件系统元数据损坏（`fsck` 报错）
 - 用户数据损坏（SHA-256 校验和不匹配）
 - SMART 介质错误增加
- 通过以上四项检查全面验证 SSD 在操作系统中断场景下的稳定性

关键设计决策：

1. **次级驱动器假设**：OSINT 测试假设待测 SSD 是次级驱动器（非系统盘），系统盘需要支持 S3/S4 休眠唤醒。待测盘上创建独立分区和文件系统用于测试
2. **活跃 IO 休眠**：默认在 `fio` 活跃读写时触发休眠，模拟真实业务场景下系统休眠的情况；支持 `--osint-io-idle` 切换为空闲状态休眠
3. **rtcwake 全自动唤醒**：使用 `rtcwake` 设置 RTC 闹钟定时唤醒，无需人工按电源键，适合长期循环自动化测试
4. **四项唤醒后检查**：磁盘标识 + 分区/文件系统 + 数据完整性 + SMART，全面覆盖操作系统中断可能导致的各类故障
5. **最终功能测试**：全部循环后执行容量 + SMART + 顺序读写性能，确认长期休眠唤醒循环后 SSD 功能正常、性能无衰减

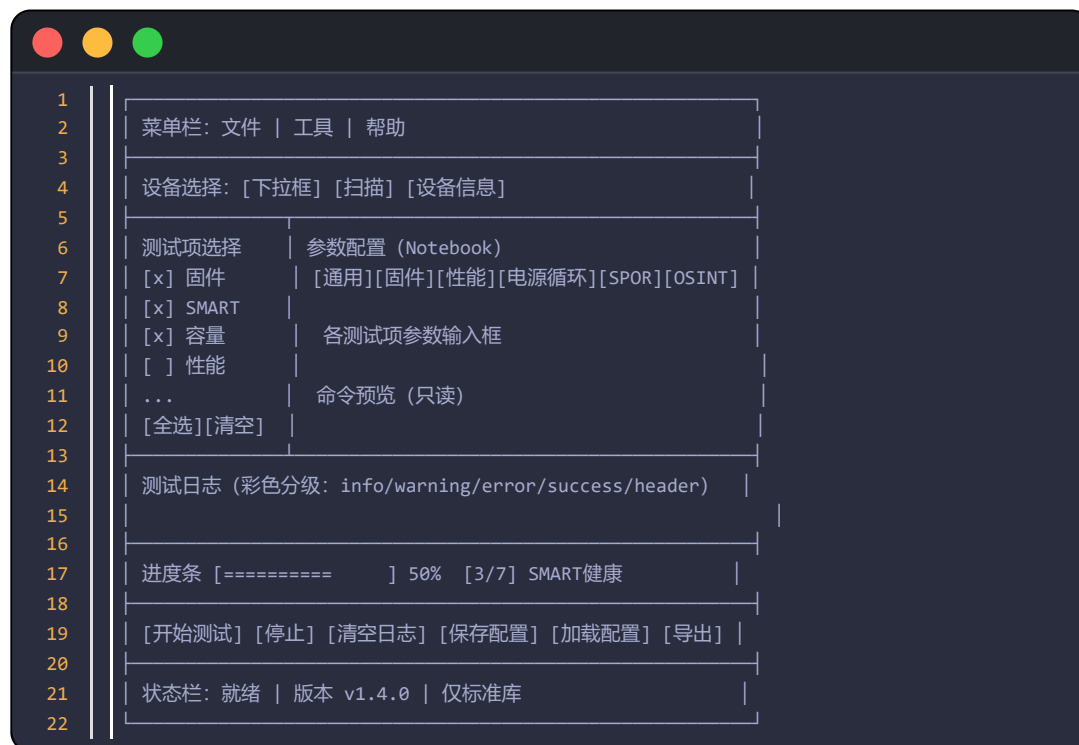
3.2.9 可视化上位机（GUI）

设计原则：

- GUI 与核心测试逻辑解耦：GUI 通过 `subprocess.Popen` 调用命令行模式执行测试，不直接调用内部类，确保命令行和 GUI 两种模式行为完全一致

- 仅依赖标准库：使用 Python 自带的 tkinter，无需安装 PyQt/PySide 等第三方库
- 实时反馈：后台线程读取子进程 stdout，通过 `root.after()` 线程安全地更新 UI

界面布局：



Fence 6

核心功能：

1. **设备自动扫描**：`lsblk -J` + `nvme list -o json` 双源扫描，自动过滤 loop/ram/zram/sr 设备，显示型号/容量/接口/SN/FW
2. **测试项勾选**：7 个测试项独立复选框，支持全选/清空，默认勾选 SMART+容量（非破坏性）
3. **参数可视化配置**：6 个 Notebook 标签页（通用/固件/性能/电源循环/SPOR/OSINT），每个参数对应输入框/下拉框/复选框，实时命令预览
4. **实时日志**：子进程 stdout 逐行读取，按关键字自动着色（error=红/warning=橙/success=绿/header=蓝），自动滚动到底部
5. **进度追踪**：从日志解析 `[x/y]` 开始测试 模式，计算整体进度百分比，显示当前测试项名称
6. **一键停止**：`terminate()` → 等待 5s → `kill()` 两级停止，确保子进程被正确终止
7. **配置持久化**：保存/加载 JSON 配置文件，包含设备、测试项、所有参数，方便重复测试
8. **日志导出**：将实时日志导出为 txt 文件，便于离线分析
9. **权限自动处理**：非 root 启动时自动检测，提示使用 pkexec 提权执行测试

线程安全设计：

- 子进程输出读取在独立守护线程中执行
- 所有 UI 更新通过 `root.after(0, callback)` 调度到 Tk 主线程，避免跨线程操作 UI 控件
- `is_running` 标志位控制开始/停止按钮状态

关键设计决策：

1. **subprocess 调用而非直接调用**：GUI 通过命令行调用核心逻辑，而非直接 import 调用 `main()`。好处：①两种模式行为完全一致，避免 GUI 特有 bug；②子进程崩溃不影响 GUI 主进程；③停止测试时可直接 `terminate` 整个进程树；④命令预览直接显示实际执行命令，便于调试

- 2. **tkinter 而非 PyQt**: tkinter 是 Python 标准库自带, Ubuntu 默认已安装 (python3-tk), 无需额外依赖, 符合脚本"仅标准库"的设计原则
- 3. **参数与命令行一一对应**: GUI 中每个参数控件直接映射到命令行参数, `build_command()` 方法构建完整命令列表, 命令预览框实时显示, 用户可清楚知道实际执行的命令
- 4. **默认勾选非破坏性测试**: 启动时默认勾选 SMART 和容量 (只读非破坏性), 性能/固件/电源循环/SPOR/OSINT 默认不勾选, 避免误操作导致数据丢失

3.2.10 3.2.10 读/写测试 (Read/Write Test)

与性能测试的核心区别:

- 性能测试 (Performance): 测量 SSD 的性能指标 (带宽/IOPS/延迟/QoS), 关注"有多快"
- 读/写测试 (Read/Write): 验证 SSD 数据读写功能正常和数据完整性, 关注"数据是否完好", 通过 fio verify 机制逐块校验写入和读取的数据一致性

三种测试模式:

模式 1: 全磁盘写入 + 读取验证 (full_disk)

- 阶段 1: 使用 fio 对整个磁盘执行顺序写入, `--verify=md5` 让 fio 在每个 block 头部计算并存储 MD5 校验和 (`--do_verify=0` 写入阶段不验证)
- 阶段 2: 对整个磁盘执行顺序读取, `--do_verify=1` 让 fio 读取每个 block 时重新计算 MD5 并与存储的校验和比对, 不一致则报错 (`--verify_fatal=1` 立即终止, `--verify_dump=1` 转储错误块)
- 验证全磁盘范围无数据丢失、损坏或比特错误

模式 2: 多文件大小重复读写周期 (file_cycle)

- 支持 5 种常用文件大小: 256MB、1GB、4GB、16GB、32GB (GUI 中可自由勾选)
- 对每个选定的文件大小, 执行指定次数 (`--rw-cycles`, 默认 3) 的写→读验证循环
- 使用 `--size=X M` 限定 fio 测试范围, `--do_verify=1` 写入后立即读取验证
- 自动跳过超过设备容量的文件大小
- 每轮后检查设备是否在线 (掉盘检测)
- 全部完成后检查 SMART 介质错误数

模式 3: 24小时长期读写验证 (long_run)

- 阶段 1: 先写入 32GB 基准测试数据 (带 verify 校验和)
- 阶段 2: 使用 `randrw` 混合读写 (默认 70%读/30%写) 持续运行指定小时数 (`--rw-long-hours`, 默认 24), `--time_based` 基于时间运行
- 阶段 3: 长期运行结束后, 重新读取 32GB 基准数据并验证校验和, 确认长期高负载读写后数据无损坏
- 最终检查设备在线状态和 SMART 健康

fio verify 数据完整性机制:

- fio 的 verify 功能在写入时, 对每个 block 的数据计算校验和 (md5/sha256/crc32), 并将校验和存储在 block 头部的元数据区域
- 读取验证时, fio 重新计算读取数据的校验和, 与存储的校验和比对
- 如果不一致: `--verify_fatal=1` 立即终止测试并报错; `--verify_dump=1` 将错误 block 的原始数据和期望数据转储到文件, 便于分析
- 这种机制可以检测到: 比特翻转、写入失败、读取错误、数据损坏、FTL 映射错误等

设备脱机检测：

- 每轮测试前后检查 `/dev/nvmeXn1` 设备节点是否存在
- 使用 `blockdev --getsize64` 确认设备可正常读取容量
- 设备脱机（掉盘）时立即终止测试并报告失败

关键设计决策：

- fio verify 而非手动校验：**使用 fio 内置的 verify 机制（md5/sha256/crc32），而非自己写数据再读出来算 SHA-256。fio verify 在 block 级别校验，能精确定位错误 LBA，且性能更高（校验和计算在 fio 内部完成）
- 写入和验证分阶段：**全磁盘模式分写入阶段（`--do_verify=0`）和读取验证阶段（`--do_verify=1`），而非写入时立即验证。这样更接近"先写满盘，再全盘读校验"的标准测试流程
- 文件大小可自由选择：**GUI 中 256MB/1GB/4GB/16GB/32GB 五个复选框可自由勾选，命令行用 `--rw-file-sizes` 多值参数，满足不同测试场景需求
- 长期运行先写基准数据：**24小时模式先写入 32GB 基准数据（带校验和），然后长期混合读写，最后重新验证基准数据。这样可以检测长期高负载运行后，已写入的数据是否被意外损坏（如 FTL 映射错误、写入干扰等）
- 自动跳过超容量文件大小：**文件周期模式中，如果选定的文件大小超过设备容量，自动跳过并记录警告，避免 fio 因空间不足而失败

3.3 测试结果数据结构

每条测试结果包含：

```
1  {
2      "test_item": "firmware_upgrade",      # 测试项标识
3      "test_name": "现场固件升级",          # 测试项名称
4      "device": "/dev/nvme0n1",             # 待测设备
5      "start_time": "2026-09-07 10:00:00",  # 开始时间
6      "end_time": "2026-09-07 10:05:00",   # 结束时间
7      "duration_sec": 300,                  # 耗时
8      "status": "PASS",                     # PASS / FAIL / ERROR / SKIP
9      "details": { ... },                  # 详细测试数据
10     "error_message": null                 # 失败时的错误信息
11 }
```

Fence 7

3.4 日志与报告

- 日志：`logs/ssd_test_YYYYMMDD_HHMMSS.log`，记录所有执行命令、输出摘要、测试进度
- JSON 报告：`reports/ssd_test_report_YYYYMMDD_HHMMSS.json`，完整结构化结果
- 文本摘要：控制台实时输出 + 报告文件末尾的 PASS/FAIL 汇总



4. 四、脚本使用步骤

4.1 脚本获取与放置

将 `ssd_test.py` 放置到任意工作目录，例如：

```
1 | mkdir -p ~/ssd_test
2 | cp ssd_test.py ~/ssd_test/
3 | cd ~/ssd_test
4 | chmod +x ssd_test.py
```

Fence 8

4.2 命令行参数说明

```
1 | sudo python3 ssd_test.py [选项]
```

Fence 9

参数	缩写	说明	默认值
<code>--device</code>	<code>-d</code>	待测设备路径（必填），如 <code>/dev/nvme0n1</code>	无
<code>--test</code>	<code>-t</code>	指定测试项，可多选： <code>fw/smart/capacity/perf/all</code>	<code>all</code>
<code>--fw-image</code>		固件镜像文件路径（fw 测试必填）	无
<code>--fw-action</code>		固件操作类型： <code>upgrade/downgrade</code>	<code>upgrade</code>
<code>--fw-slot</code>		固件槽位（1-7），默认自动选择	自动
<code>--perf-state</code>		性能测试状态： <code>fob/steady/both</code>	<code>both</code>
<code>--perf-runtime</code>		单项 fio 运行时长（秒）	60
<code>--perf-qd</code>		性能测试队列深度	32
<code>--perf-bs</code>		性能测试块大小（如 4k, 128k）	自动
<code>--precondition</code>		稳态预处理开关： <code>on/off</code>	<code>on</code>
<code>--pc-cycles</code>		电源循环测试循环次数	10
<code>--pc-power-mode</code>		电源控制模式： <code>ipmi/manual</code>	<code>ipmi</code>
<code>--ipmi-host</code>		IPMI BMC 地址（ipmi 模式必填）	无
<code>--ipmi-user</code>		IPMI 用户名	ADMIN
<code>--ipmi-pass</code>		IPMI 密码	ADMIN
<code>--pc-mount-point</code>		电源循环测试挂载点	<code>/mnt/ssd_test</code>
<code>--pc-state-file</code>		电源循环状态文件路径	<code>/var/lib/ssd_power_cycle_state.json</code>
<code>--pc-boot-timeout</code>		开机等待超时（秒）	300
<code>--pc-off-interval</code>		断电后等待上电间隔（秒）	30
<code>--pc-rw-duration</code>		每循环混合读写运行时长（秒）	120

Table 2

参数	缩写	说明	默认值
--spor-cycles		SPOR 意外断电循环次数	10
--spor-delay		SPOR 写入后触发硬件断电的等待秒数	5
--spor-test-size		SPOR 测试数据大小（GB）	20
--spor-lba-size		SPOR LBA 大小（字节）	4096
--spor-skip-lba		掉电边界跳过验证的 LBA 数	8
--spor-poweroff-delay-ms		Timeboard 硬件断电延时（毫秒，越短越意外）	500
--spor-mixed-rw		SPOR 使用混合读写负载（默认纯顺序写）	关
--spor-mixed-read-ratio		混合读写模式下读占比（%）	70
--spor-no-final-test		跳过 SPOR 最终完整功能测试	关
--spor-timeboard-port		Timeboard 串口设备路径	/dev/ttyUSB0
--spor-state-file		SPOR 状态文件路径	/var/lib/ssd_spor_state.json
--osint-cycles		OSINT 测试循环次数	10
--osint-sleep-type		OSINT 休眠类型: s3/s4/both	s3
--osint-sleep-duration		OSINT 每次休眠持续秒数（rtcwake 定时唤醒）	30
--osint-io-idle		OSINT 空闲状态休眠（不启动 IO 负载，默认活跃 IO 模式）	关
--osint-io-duration		OSINT 每轮休眠前持续 IO 秒数	60
--osint-mount-point		OSINT 测试挂载点	/mnt/ssd_osint
--osint-state-file		OSINT 状态文件路径	/var/lib/ssd_osint_state.json
--rw-mode		读/写测试模式: full_disk/file_cycle/long_run/all	all
--rw-file-sizes		文件周期测试文件大小（可多选）: 256MB/1GB/4GB/16GB/32GB	全部
--rw-cycles		文件周期每个大小循环次数	3
--rw-pattern		写入数据 pattern: random/0xAA/0x55/0x00/0xFF	random
--rw-verify		数据校验方式: md5/sha256/crc32	md5
--rw-long-hours		长期运行测试小时数	24
--rw-block-size		读写块大小	4k
--rw-iodepth		队列深度	32

参数	缩写	说明	默认值
--rw-numjobs		并发 job 数	4
--rw-mixed-read-ratio		长期运行混合读写读比例(%)	70
--output-dir	-o	报告输出目录	./reports
--log-dir		日志输出目录	./logs
--dry-run		试运行模式，只打印命令不执行	off
--yes	-y	跳过数据销毁确认提示	off
--verbose	-v	详细日志输出	off
--gui		启动可视化上位机（GUI模式）	off
--version		显示版本信息并退出	-

4.3 4.3 使用示例

4.3.1 示例 0：启动可视化上位机（GUI）

```
1 | # 推荐：root 启动 GUI，执行测试时无需再次提权
2 | sudo python3 ssd_test.py --gui
3 |
4 | # 非 root 启动 GUI，执行测试时自动 pkexec 提权
5 | python3 ssd_test.py --gui
```

Fence 10

4.3.2 示例 1：运行全部测试项（最常用）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t all \
2 |     --fw-image /path/to/firmware.bin \
3 |     --perf-state both \
4 |     --perf-runtime 120 \
5 |     -y
```

Fence 11

4.3.3 示例 2：仅运行固件升级测试

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t fw \
2 |     --fw-image /path/to/new_firmware.bin \
3 |     --fw-action upgrade
```

Fence 12

4.3.4 示例 3：仅运行固件降级测试

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t fw \
2 |     --fw-image /path/to/old_firmware.bin \
3 |     --fw-action downgrade
```

Fence 13

4.3.5 示例 4：仅运行 SMART 健康信息测试

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t smart
```

Fence 14

4.3.6 示例 5：仅运行设备容量测试

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t capacity
```

Fence 15

4.3.7 示例 6：仅运行 FOB 状态性能测试（跳过稳态预处理）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t perf \  
2 | --perf-state fob \  
3 | --perf-runtime 60 \  
4 | -y
```

Fence 16

4.3.8 示例 7：仅运行稳态性能测试（含预处理）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t perf \  
2 | --perf-state steady \  
3 | --precondition on \  
4 | --perf-runtime 300 \  
5 | -y
```

Fence 17

4.3.9 示例 8：电源循环测试（IPMI 远程电源控制，全自动）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t powercycle \  
2 | --pc-cycles 20 \  
3 | --pc-power-mode ipmi \  
4 | --ipmi-host 192.168.1.100 \  
5 | --ipmi-user ADMIN \  
6 | --ipmi-pass mypassword \  
7 | --pc-rw-duration 120 \  
8 | -y
```

Fence 18

4.3.10 示例 9：电源循环测试（手动断电上电模式）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t powercycle \  
2 | --pc-cycles 10 \  
3 | --pc-power-mode manual \  
4 | -y  
5 | # 脚本会在每次关机前提示用户手动断电，30秒后手动上电
```

Fence 19

4.3.11 示例 10: SPOR 意外电源循环测试 (Timeboard 硬件断电, 纯写模式)

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t spor \  
2 |     --spor-cycles 20 \  
3 |     --spor-delay 5 \  
4 |     --spor-poweroff-delay-ms 500 \  
5 |     --spor-timeboard-port /dev/ttyUSB0 \  
6 |     -y  
7 | # 在 fio 活跃写入 5 秒后, Timeboard 硬件在 500ms 后直接切断 SSD 电源 (不 sync、不待机)  
8 | # 上电后自动验证前段 pattern22 + 后段 pattern11 + SMART, 循环 20 次
```

Fence 20

4.3.12 示例 11: SPOR 测试 (混合读写模式, 高压场景)

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t spor \  
2 |     --spor-cycles 10 \  
3 |     --spor-mixed-rw \  
4 |     --spor-mixed-read-ratio 70 \  
5 |     --spor-test-size 20 \  
6 |     -y  
7 | # 使用 randrw 混合读写 (70%读+30%写, 4K随机), 更接近真实业务负载  
8 | # 混合读写对 FTL 映射表更新压力更大, 更能暴露 PLP 缺陷
```

Fence 21

4.3.13 示例 12: OSINT 操作系统中断测试 (S3 休眠, 活跃 IO 模式)

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t osint \  
2 |     --osint-cycles 20 \  
3 |     --osint-sleep-type s3 \  
4 |     --osint-sleep-duration 30 \  
5 |     --osint-io-duration 60 \  
6 |     -y  
7 | # fio 混合读写运行 60 秒后, 在活跃 IO 时触发 S3 休眠  
8 | # rtcwake 设置 RTC 闹钟 30 秒后自动唤醒, 无需人工干预  
9 | # 唤醒后检查磁盘标识/分区/文件系统/数据完整性/SMART, 循环 20 次
```

Fence 22

4.3.14 示例 13: OSINT 测试 (S3+S4 交替, 空闲模式)

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t osint \  
2 |     --osint-cycles 10 \  
3 |     --osint-sleep-type both \  
4 |     --osint-sleep-duration 60 \  
5 |     --osint-io-idle \  
6 |     -y  
7 | # 每轮交替执行 S3 和 S4 休眠, 空闲状态下触发 (不启动 IO 负载)  
8 | # S4 休眠需要系统配置足够大的 swap 分区
```

Fence 23

4.3.15 示例 14: 读/写测试 (全磁盘写入+读取验证)

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t rw \  
2 |     --rw-mode full_disk \  
3 |     --rw-verify md5 \  
4 |     --rw-pattern random \  
5 |     -y  
6 | # 全磁盘顺序写入 (每块带MD5校验和), 然后全磁盘读取验证  
7 | # 验证整个磁盘范围无数据丢失、损坏或比特错误  
8 | # 耗时取决于磁盘容量和写入速度 (1TB约10-20分钟)
```

4.3.16 示例 15：读/写测试（多文件大小重复读写周期）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t rw \
2 |     --rw-mode file_cycle \
3 |     --rw-file-sizes 256MB 1GB 4GB 16GB 32GB \
4 |     --rw-cycles 5 \
5 |     --rw-verify sha256 \
6 |     -y
7 | # 对 256MB/1GB/4GB/16GB/32GB 五种文件大小，每种执行 5 轮写→读验证
8 | # 使用 SHA-256 校验（比 MD5 更严格，速度稍慢）
9 | # 自动跳过超过设备容量的文件大小
```

Fence 25

4.3.17 示例 16：读/写测试（24小时长期读写验证）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t rw \
2 |     --rw-mode long_run \
3 |     --rw-long-hours 24 \
4 |     --rw-mixed-read-ratio 70 \
5 |     --rw-block-size 4k \
6 |     --rw-iodepth 32 \
7 |     -y
8 | # 先写入32GB基准数据（带校验和），然后70%读+30%写混合负载持续运行24小时
9 | # 运行结束后重新读取32GB基准数据并验证校验和
10 | # 验证长期高负载读写后数据无损坏、设备无脱机
```

Fence 26

4.3.18 示例 17：读/写测试（全部模式，GUI 中也可配置）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t rw --rw-mode all -y
2 | # 依次执行：全磁盘验证 → 多文件大小周期 → 24小时长期运行
3 | # 完整覆盖读/写测试的所有场景
```

Fence 27

4.3.19 示例 18：试运行模式（不实际执行，仅验证命令和参数）

```
1 | sudo python3 ssd_test.py -d /dev/nvme0n1 -t all --dry-run
```

Fence 28

4.3.20 示例 19：SATA SSD 测试

```
1 | sudo python3 ssd_test.py -d /dev/sda -t smart,capacity -y
```

Fence 29

4.4 4.4 输出说明

4.4.1 控制台输出

脚本运行时实时输出测试进度：

```
1 | [2026-09-07 10:00:00] [INFO] =====
2 | [2026-09-07 10:00:00] [INFO] SSD 自动化测试脚本启动
3 | [2026-09-07 10:00:00] [INFO] 设备: /dev/nvme0n1
4 | [2026-09-07 10:00:00] [INFO] 测试项: all
5 | [2026-09-07 10:00:00] [INFO] =====
6 | [2026-09-07 10:00:01] [INFO] [1/5] 设备容量测试 ...
```

```
7 [2026-09-07 10:00:02] [INFO] NVMe 总容量: 1024.2 GB (953.9 GiB)
8 [2026-09-07 10:00:02] [INFO] lsblk 容量: 1024.2 GB (953.9 GiB)
9 [2026-09-07 10:00:02] [PASS] 设备容量测试 - PASS (1.2s)
10 ...
11 [2026-09-07 10:30:00] [INFO] =====
12 [2026-09-07 10:30:00] [INFO] 测试汇总
13 [2026-09-07 10:30:00] [INFO] 设备容量: PASS
14 [2026-09-07 10:30:00] [INFO] SMART健康信息: PASS
15 [2026-09-07 10:30:00] [INFO] 固件升级: PASS
16 [2026-09-07 10:30:00] [INFO] 完整性特征: PASS
17 [2026-09-07 10:30:00] [INFO] 正常电源循环: PASS
18 [2026-09-07 10:30:00] [INFO] 总计: 5 PASS / 0 FAIL / 0 ERROR
19 [2026-09-07 10:30:00] [INFO] 报告: reports/ssd_test_report_20260907_100000.json
20 [2026-09-07 10:30:00] [INFO] 日志: logs/ssd_test_20260907_100000.log
21 [2026-09-07 10:30:00] [INFO] =====
```

4.4.2 报告文件

- JSON 报告: `reports/ssd_test_report_YYYYMMDD_HHMMSS.json`，包含所有测试项的完整结构化数据
- 日志文件: `logs/ssd_test_YYYYMMDD_HHMMSS.log`，完整命令执行记录

4.5 典型测试流程

- 准备阶段:** 确认待测设备、备份数据、卸载分区、准备固件镜像、配置 IPMI（电源循环测试需要）
- 容量测试:** 读取并校验设备容量（最快，先确认设备基本识别）
- SMART 测试:** 读取初始 SMART → 基本 R/W → 复检 SMART
- 固件测试:** 读取当前版本 → 下载/提交固件 → 复位等待 → 版本校验 → 功能校验
- 性能测试:** FOB 状态测试 → 稳态预处理 → 稳态测试 → 性能对比
- 电源循环测试:** 创建分区/文件系统 → 写入校验数据 → 混合读写负载 → 正常关机断电 → 上电开机 → 磁盘/分区/文件系统/数据检查 → 循环 N 次 → 最终完整功能测试
- 报告生成:** 汇总所有结果，生成 JSON 报告和文本摘要

5. 五、注意事项与常见问题

5.1 安全注意

- 数据销毁:** 性能测试（尤其稳态预处理）、固件升级和电源循环测试会破坏全盘数据，务必确认设备无重要数据
- 设备选择:** `--device` 参数务必确认是待测 SSD，误填系统盘（如 `/dev/sda`）会导致系统损坏
- 固件兼容性:** 固件镜像必须与设备型号匹配，错误的固件可能导致设备变砖
- 断电风险:** 固件烧写过程中严禁断电，建议使用 UPS
- 电源循环测试系统盘风险:** 电源循环测试会反复关机断电，**测试脚本和系统必须运行在非待测盘上**（如系统盘 `/dev/sda`，待测盘 `/dev/nvme0n1`），否则脚本会随系统关机中断
- IPMI 密码安全:** `--ipmi-pass` 参数会出现在进程列表和日志中，生产环境建议使用 IPMI 配置文件或环境变量
- 手动断电模式:** `--pc-power-mode manual` 时，脚本关机后需用户在 30 秒后手动上电，否则脚本无法恢复执行

8. **SPOR 硬件断电风险**：SPOR 测试通过 Timeboard 继电器直接切断 SSD 供电，**不经过系统关机**，运行 SPOR 测试时系统盘必须与待测 SSD 独立（系统盘走主板供电，待测 SSD 走 Timeboard 控制的独立电源），否则会导致系统异常断电损坏系统盘数据
9. **SPOR 测试需配置开机自启**：SPOR 阶段1触发硬件断电后脚本进程随系统终止，上电后需通过 systemd service 或桌面自启（.desktop）自动运行脚本，脚本检测到状态文件 phase=2 后自动恢复执行阶段2
10. **Timeboard 串口权限**：Timeboard 通常使用 `/dev/ttyUSB0`，需确保当前用户有 dialout 组权限或执行

```
sudo chmod 666 /dev/ttyUSB0
```
11. **OSINT 休眠会中断当前会话**：OSINT 测试触发 S3/S4 休眠时，当前 SSH 会话或桌面会话会被中断。建议通过 systemd service 或 nohup 后台运行脚本，避免会话中断导致脚本终止
12. **S4 休眠需要 swap 分区**：S4（挂起到磁盘）需要系统配置至少大于内存大小的 swap 分区，否则休眠会失败。使用 `swapon --show` 确认 swap 配置
13. **rtcwake 需要 BIOS 支持 RTC 唤醒**：部分主板 BIOS 默认禁用 RTC 唤醒，需在 BIOS 设置中启用 "RTC Alarm" 或 "Wake on RTC"，否则 rtcwake 设置的闹钟无法唤醒系统
14. **OSINT 待测盘应为次级驱动器**：OSINT 测试在待测盘上创建分区和文件系统，系统盘必须独立于待测盘，且系统盘需支持 S3/S4 休眠唤醒
15. **GUI 需图形显示环境**：`--gui` 模式需要 X11/Wayland 显示环境，SSH 远程无显示环境下请使用命令行模式，或配置 X11 转发（`ssh -X`）
16. **GUI 推荐 root 启动**：虽然 GUI 支持非 root 启动并自动 pkexec 提权，但推荐 `sudo python3 ssd_test.py --gui` 启动，避免每次执行测试都弹出密码提示
17. **GUI 停止测试的影响**：点击"停止"会 terminate 子进程，正在执行的 fio/断电等操作会被中断，可能导致磁盘处于不一致状态，建议停止后运行一次 SMART 检查
18. **GUI 配置文件兼容性**：保存的 JSON 配置文件包含所有参数，可在不同机器间复用，但设备路径可能不同，加载后需重新选择设备
19. **RW 全磁盘测试会清除所有数据**：读/写测试的 full_disk 模式会对整个磁盘执行写入，磁盘上所有数据将被永久清除且不可恢复，执行前务必备份重要数据
20. **RW 全磁盘测试耗时较长**：1TB 磁盘全磁盘写入+读取验证约需 10-20 分钟（取决于写入速度），4TB 磁盘可能需要 1 小时以上，请合理安排测试时间
21. **RW 24小时长期测试不可中断**：long_run 模式设计为连续运行 24 小时，中途停止会导致基准数据验证不完整。如需停止，请确保在测试结束后重新运行一次完整验证
22. **RW fio verify 校验和存储开销**：fio verify 模式在每个 block 头部存储校验和（MD5 为 16 字节），会略微减少实际可用数据空间，但不影响测试结果的准确性
23. **RW 文件大小选择建议**：小文件（256MB/1GB）适合快速验证基本读写功能；大文件（16GB/32GB）更能暴露 SSD 在大跨度 LBA 访问时的 FTL 映射问题，建议至少包含 4GB 和 16GB 两种大小

5.2 5.2 常见问题

问题	可能原因	解决方案
<code>nvme: command not found</code>	未安装 nvme-cli	<code>sudo apt install nvme-cli</code>
<code>Permission denied</code>	未使用 root 权限	命令前加 <code>sudo</code>

Table 3

问题	可能原因	解决方案
固件 commit 后设备消失	设备复位中，正常现象	脚本会自动轮询等待，最长 120s
fio 报 device is busy	设备有分区已挂载	<code>sudo umount /dev/nvme0n1p*</code>
SMART 温度显示异常	部分厂商传感器读取方式不同	脚本同时尝试 <code>nvme smart-log</code> 和 <code>smartctl</code>
容量显示与标称不符	厂商十进制 vs OS 二进制，且有保留空间	正常现象，报告中同时标注 GB 和 GiB
稳态预处理时间过长	大容量盘预处理耗时较长	可通过 <code>--precondition off</code> 跳过，或减小盘容量测试
电源循环测试关机后脚本不恢复	脚本运行在待测盘上，关机后脚本丢失	确保脚本和系统运行在非待测盘上；配置 <code>systemd</code> 开机自启
IPMI 连接失败	BMC 地址/账号错误、网络不通	<code>ping <BMC_IP></code> 测试连通性；确认 IPMI 账号权限；检查防火墙
开机后设备未枚举	SSD 掉盘或上电初始化慢	脚本会轮询等待最长 300s；检查 SSD 供电和数据线；查看 <code>dmesg</code>
数据完整性校验失败	电源循环导致数据丢失或文件系统损坏	本循环标记 FAIL；检查文件系统日志（ <code>dmesg/journalctl</code> ）；确认 SSD 是否有掉电保护
手动模式下不知道何时上电	脚本关机后会打印提示	看到 请在 30 秒后手动上电 提示后，等待 30 秒再按电源键
SPOR 测试后系统也断电了	系统盘和待测 SSD 接在同一电源上，Timeboard 断电时系统也掉电	确保系统盘走主板供电，待测 SSD 走 Timeboard 控制的独立电源通道
Timeboard 连接失败	串口权限不足或串口设备名不对	<code>sudo chmod 666 /dev/ttyUSB0</code> ；检查 <code>ls /dev/ttyUSB* /dev/ttyACM*</code> 确认设备名；用 <code>--spor-timeboard-port</code> 指定正确端口
SPOR 上电后脚本不自动恢复	未配置开机自启，或状态文件损坏	配置 <code>systemd service</code> 或桌面 <code>.desktop</code> 自启；检查 <code>/var/lib/ssd_spor_state.json</code> 是否存在且 <code>phase=2</code>
SPOR pattern22 验证总是失败	PLP 失效，或掉电边界跳过 LBA 数不够	增加 <code>--spor-skip-lba</code> 到 16 或 32；如果仍失败，说明 SSD PLP 可能无法在该负载下保护 FTL 元数据
SPOR 写入量偏少	<code>iodepth</code> 或 <code>numjobs</code> 太低，或 <code>delay</code> 太短	默认已使用 <code>iodepth=32 numjobs=4</code> ；可增加 <code>--spor-delay</code> 到 10-30 秒增加写入量
fio iolog 最后一行截断	掉电时 <code>iolog</code> 正在写入，最后一行不完整	正常现象，脚本通过 LBA 对齐校验自动跳过非对齐条目
OSINT 休眠后系统不唤醒	BIOS 禁用了 RTC 唤醒，或 <code>rtcwake</code> 权限不足	在 BIOS 中启用 "RTC Alarm"/"Wake on RTC"；确认以 <code>root</code> 运行；使用 <code>sudo rtcwake -m mem -s 10</code> 手动测试
S4 休眠失败	swap 分区不足或未配置	<code>swapon --show</code> 确认 swap；确保 swap 大小 $\geq$ 内存大小； <code>sudo falldate -l 16G /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile</code>

问题	可能原因	解决方案
OSINT 休眠后 SSH 断开	休眠导致网络中断，SSH 会话终止	使用 <code>nohup sudo python3 ssd_test.py ... &</code> 后台运行，或配置 <code>systemd service</code> 自动运行
OSINT 数据完整性校验失败	休眠唤醒过程中 SSD 固件异常，或文件系统未正确同步	检查 <code>dmesg/journalctl</code> 中是否有 NVMe 错误；确认休眠前 <code>sync</code> 正常执行；增加 <code>--osint-io-duration</code> 确保数据充分落盘
OSINT 磁盘标识变化	休眠唤醒后 <code>/dev/disk/by-id/</code> 链接丢失或设备名变化	检查 <code>udev</code> 规则；确认 SSD 固件支持休眠唤醒；记录每轮磁盘标识变化情况用于分析
GUI 启动报错 <code>no display name</code>	无图形显示环境（SSH 纯终端）	使用命令行模式；或配置 X11 转发 <code>ssh -X user@host</code> ；或在本地桌面环境运行
GUI 启动报错 <code>No module named tkinter</code>	<code>python3-tk</code> 未安装	<code>sudo apt-get install python3-tk</code>
GUI 中扫描不到设备	<code>lsblk/nvme-cli</code> 未安装，或设备未被系统识别	<code>sudo apt-get install nvme-cli smartmontools</code> ； <code>lsblk</code> 确认设备存在；检查设备连接
GUI 执行测试时卡住无日志	子进程输出缓冲，或测试项本身耗时长	确认设置了 <code>PYTHONUNBUFFERED=1</code> （脚本自动设置）；性能测试/稳态预处理可能耗时数十分钟，属正常现象
GUI 停止测试后设备无法访问	停止时 <code>fio/</code> 写入被中断，文件系统可能处于不一致状态	运行 <code>sudo fsck /dev/nvme0n1p1</code> 检查修复；重新格式化测试分区；运行 SMART 检查确认硬件无异常
RW 全磁盘验证失败 <code>verify mismatch</code>	读取数据与写入时校验和不一致，存在比特错误或数据损坏	检查 <code>dmesg</code> 中是否有 NVMe 错误；确认 SSD 固件是否最新；更换数据线/PCIe 插槽重试；如持续失败可能是 NAND 硬件故障
RW <code>fio</code> 报错 <code>No space left on device</code>	文件大小超过设备容量，或全磁盘模式下分区占用了空间	文件周期模式会自动跳过超容量大小；全磁盘模式请确保直接使用裸设备（ <code>/dev/nvme0n1</code> ）而非分区（ <code>/dev/nvme0n1p1</code> ）
RW 24小时测试中途设备脱机	长期高负载下 SSD 过热或固件异常导致掉盘	检查 SSD 温度（SMART <code>temperature</code> ）；改善散热；检查 <code>dmesg/journalctl</code> 中的 NVMe 错误；更新 SSD 固件
RW md5 与 sha256 如何选择	md5 速度快但理论存在碰撞风险；sha256 更安全但速度稍慢	常规测试用 md5 足够（ <code>fio verify</code> 的 md5 是块级校验，碰撞概率可忽略）；对数据完整性要求极高的场景用 sha256
RW 测试后 SMART 介质错误增加	读写过程中发现 NAND 读取错误，SSD 已通过 ECC 纠正但记录了错误	少量介质错误（个位数）在 SSD 生命周期内属正常；如果错误数快速增加或可用备件下降，说明 SSD 可能存在硬件问题，建议更换

5.3 5.3 扩展建议

- 如需支持厂商专用工具，可在 `FirmwareTester` 中添加 `vendor_tool` 分支
 - 如需 HTML 报告，可基于 JSON 报告使用 jinja2 模板生成
 - 如需多盘并行测试，可使用多进程包装，每个设备一个测试实例
 - 如需接入 CI/CD，可将 JSON 报告转换为 JUnit XML 格式
-

6. 六、版本信息

- 脚本版本：v1.5
- 适用平台：Ubuntu 20.04+
- Python 版本：3.8+
- 依赖工具：nvme-cli $\geq$ 1.9, smartmontools $\geq$ 7.0, fio $\geq$ 3.16, ipmitool $\geq$ 1.8（电源循环测试），e2fsprogs/parted（电源循环测试），pyserial $\geq$ 3.0（SPOR Timeboard 控制），rtcwake/util-linux（OSINT 休眠唤醒），python3-tk（GUI 上位机，Ubuntu 默认已安装）
- 更新记录：
 - v1.0：初始版本，支持固件升降级、SMART、容量、性能测试
 - v1.1：新增正常电源循环测试（IPMI/手动双模式、状态文件持久化、数据完整性校验、最终完整功能测试）
 - v1.2：新增意外电源循环测试 SPOR（Timeboard Modbus RTU 硬件断电、两阶段状态机、fio iolog 掉电位置追踪、前段/后段 pattern 验证、掉盘检测、SMART 检查、最终完整功能测试、混合读写支持）；修复原始 SPOR 代码缺陷（删除断电前 sync、提高 iodepth 写入压力、修复 PCIe 类代码筛选）
 - v1.3：新增操作系统中断测试 OSINT（S3/S4 休眠唤醒、rtcwake 定时唤醒、fio 混合读写持续负载、磁盘标识/分区/文件系统/SHA-256 数据完整性/SMART 四项唤醒后检查、活跃IO/空闲双模式、S3+S4 交替模式、最终完整功能测试）
 - v1.4：新增可视化上位机 GUI（tkinter 标准库、设备自动扫描、7项测试勾选、6标签页参数配置、实时彩色日志、进度追踪、一键停止、配置保存/加载JSON、日志导出、pkexec自动提权、命令预览）；支持 `-gui` 参数启动图形界面；GUI 通过 subprocess 调用命令行模式，双模式行为一致
 - v1.5：新增读/写测试 Read/Write Test（全磁盘写入+读取验证 fio verify md5/sha256/crc32、多文件大小 256MB/1GB/4GB/16GB/32GB 重复读写周期、24小时长期混合读写验证、设备在线/掉盘检测、SMART 介质错误检查、5种数据 pattern、自动跳过超容量文件大小）；GUI 新增读/写测试标签页（文件大小自由勾选、模式选择、所有参数可视化配置）；测试项扩展至 8 项